

LAB: Stepper Motor

Date: 2023-10-31

Author/Partner: Dongjun, Lee

Github: <https://github.com/henny041520-commits/EC-DJLee-042>

Demo Video:

[EC 25-2] LAB Stepper_Motor_Full_Step_10 : <https://youtu.be/f22WbnkMSMU>

[EC 25-2] LAB Stepper_Motor_Full_Step_10_reverse : <https://youtu.be/Zao6RR0aJBU>

[EC 25-2] LAB Stepper_Motor_Full_Step_min : https://youtu.be/HYBWWg_fNro

[EC 25-2] LAB Stepper_Motor_Full_Step_max : <https://youtu.be/KBXkiTyvAHE>

[EC 25-2] LAB Stepper_Motor_Half_Step_10 : https://youtu.be/M-bgF3o_06o

[EC 25-2] LAB Stepper_Motor_Half_Step_10_reverse : <https://youtu.be/rKX4f4DsPn8>

[EC 25-2] LAB Stepper_Motor_Half_Step_min : <https://youtu.be/iKn8MdpMG90>

[EC 25-2] LAB Stepper_Motor_Half_Step_max : <https://youtu.be/Z0JvAXHSYPM>

PDF version: https://github.com/henny041520-commits/EC-DJLee-042/blob/main/report/LAB_Stepper_Motor.pdf

Introduction

In this lab, we will learn how to drive a stepper motor with digital output of GPIOs of MCU. You will use a FSM to design the algorithm for stepper motor control.

You must submit

- LAB Report (*.pdf & *.md)
- Zip source files(main*.c, ecRCC.h, ecGPIO.h, ecSysTick.c etc...).
- Only the source files. Do not submit project files

Requirement

Hardware

- MCU
 - NUCLEO-F411RE
- Actuator/Sensor/Others:
 - 3Stepper Motor 28BYJ-48
 - Motor Driver A4988 (tutorial)

- Motor Driver ULN2003 (lab)
- breadboard

Software

- Keil uVision, CMSIS, EC_HAL library

Problem : Stepper Motor with 4-input sequence

For the lab, we are going to use another stepper motor driver of **ULN2003 motor driver**.

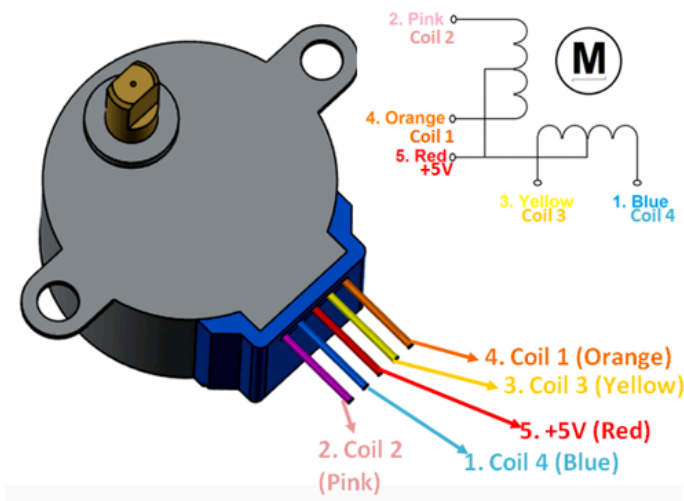
[See here for ULN2003 spec sheet](#)

Here, you have to give 4-input pulses in sequence.

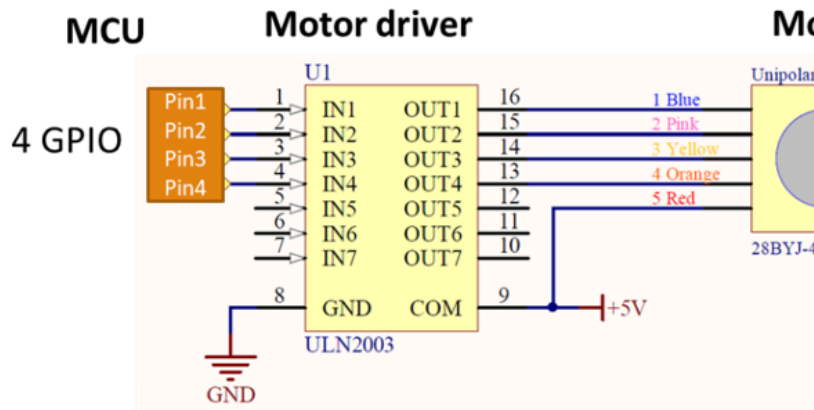
Hardware Connection

Read the specification sheet of the motor and the motor driver for wiring and min/max input voltage/current.

Unipolar Stepper Motor 28BYJ-48



- Rated Voltage: 5V DC
- Number of Phases: 4
- Stride Angle: $5.625^\circ/64$
- Gear ratio: 1/32
- Pull in torque: 300 gf.cm
- Coil: Unipolar 5 lead coil

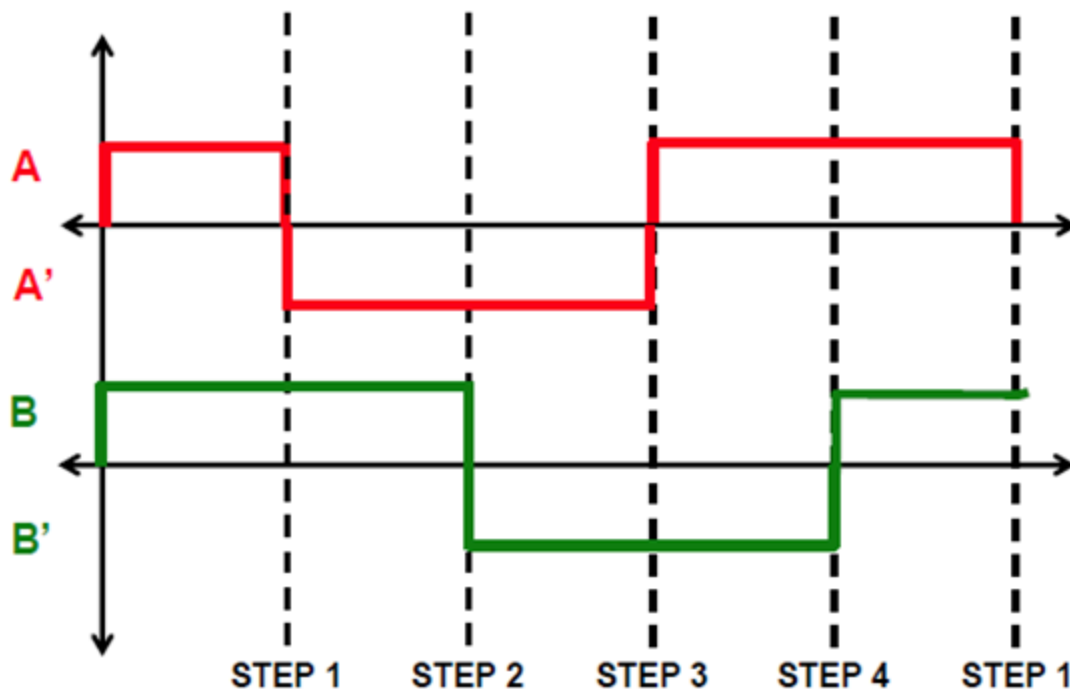


Stepper Motor Sequence

We will use unipolar stepper motor for this lab

Fill in the blanks of each output data depending on the below sequence.

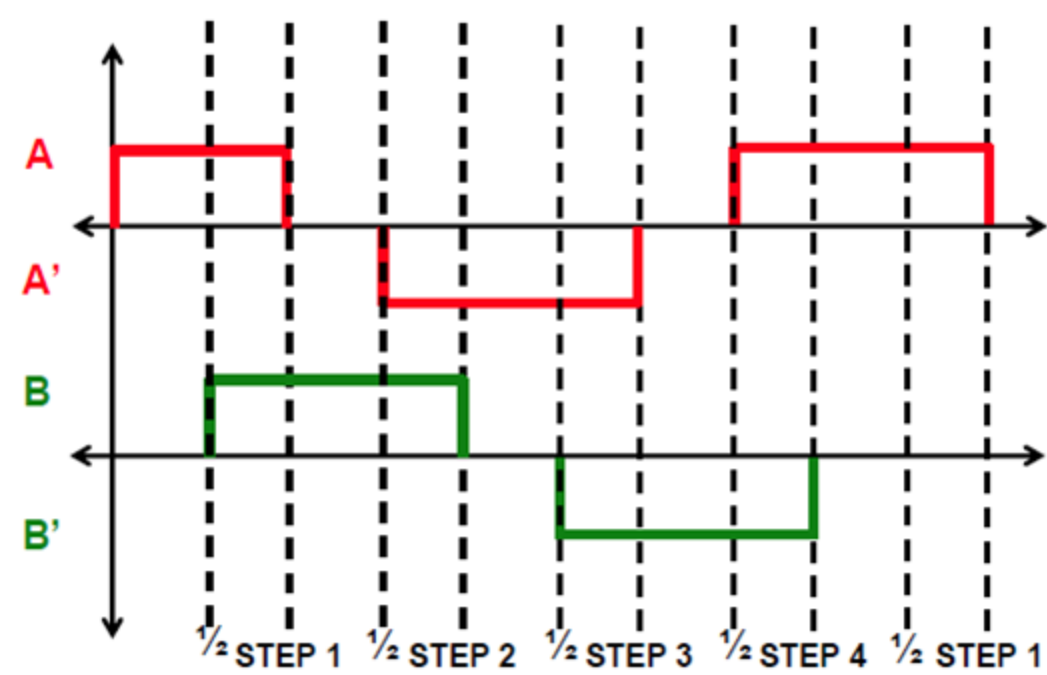
Full-stepping sequence



Phase	Port_Pin	Sequence			
		1	2	3	4
A	PB_10	H	L	L	H

Phase	Port_Pin	Sequence			
B	PB_4	H	H	L	L
A'	PB_5	L	H	H	L
B'	PB_3	L	L	H	H

Half-stepping sequence



Phase	Port_Pin	Sequence							
		1	2	3	4	5	6	7	8
A	PB_10	H	H	L	L	L	L	L	H
B	PB_4	L	H	H	H	L	L	L	L
A'	PB_5	L	L	L	H	H	H	L	L
B'	PB_3	L	L	L	L	L	H	H	H

Finite State Machine

Draw a State Table for Full-Step Sequence. You can choose either Moore FSM or Mealy.

- Full-Stepping Sequence

State	Next State		Output
	DIR=0	DIR=1	(A B A' B')

State	Next State		Output
S0	S1	S3	1,1,0,0
S1	S2	S0	0,1,1,0
S2	S3	S1	0,0,1,1
S3	S0	S2	1,0,0,1

- Half-Stepping Sequence

State	Next State		Output
	DIR=0	DIR=1	(A B A' B')
S0	S1	S7	1,0,0,0
S1	S2	S0	1,1,0,0
S2	S3	S1	0,1,0,0
S3	S4	S2	0,1,1,0
S4	S5	S3	0,0,1,0
S5	S6	S4	0,0,1,1
S6	S7	S5	0,0,0,1
S7	S0	S6	1,0,0,1

You have to program the stepping sequence using the state table. You can define the states using structures.

Example Code: Output Pins = A-B-A'-B'

```
// State number structure
typedef enum {
    S0, S1, S2, S3
} stateNum;

// State structure
typedef struct {
    stateNum next[2];
    uint8_t out;
} State_t;

// State Table Definition (Moore)
State_t FSM[4] = {
    {{S1, S3}, 0x9},    // output = b1001  A-B-A'-B'
```

```

    {{S2, S0}, 0xE},    // output = b1100
    {{S3, S1}, 0x6},    // output = b0110
    {{S0, S2}, 0x3}     // output = b0011
};

```

Create HAL library

Download files:

- [ecStepper_student.h, ecStepper_student.c](#)

Then, change the library files as ecStepper.h, ecStepper.c

Declare and define the following functions in your library.

You must update your header files located in the directory `EC \lib\`.

ecStepper.h

```

// Initialize with 4 pins
// ( A, B, AN, BN)
void Stepper_init(PinName_t A, PinName_t B, PinName_t AN, PinName_t BN);

// whatSpeed [rev/min]
void Stepper_setSpeed(long whatSpeed);

// Run for n Steps
void Stepper_step(uint32_t steps, uint32_t direction, uint32_t mode);

// Immediate Stop.
void Stepper_stop(void);

```

ecStepper.c

```

#include "stm32f4xx.h"
#include "ecStepper2.h"

// -----
// State number labels used as indices into the FSM tables.
// These integers carry no physical meaning by themselves.
// -----
#define S0 0
#define S1 1
#define S2 2
#define S3 3

```

```

#define S4 4
#define S5 5
#define S6 6
#define S7 7

// -----
// Global stepper parameters
// - direction : legacy/global flag (actual direction is passed to
Stepper_step())
// - step_delay: inter-step delay in milliseconds (set by Stepper_setSpeed())
// - step_per_rev: steps per revolution BASIS used by Stepper_setSpeed()
//               (here: 64*32 = 2048, i.e., half-step basis with gearbox)
// -----
uint32_t direction = 1; // reverse
uint32_t step_delay = 100;
uint32_t step_per_rev = 64*32;

// -----
// Stepper driver context (pins, counters, etc.), defined in ecStepper2.h
// -----
volatile Stepper_t myStepper;

// -----
// FULL stepping sequence - Mealy FSM
// Each entry encodes:
//   next[2] : next state for dir=0 and dir=1
//   out[4]  : coil pattern [A, B, a, b] where 1=energized, 0=off
// Sequence comment (right) shows typical two-phase ON pattern.
// -----
typedef struct {
    uint32_t next[2]; // next[0] for dir=0, next[1] for dir=1
    uint8_t out[4];   // output bits for coils [A, B, a, b]
} State_full_t;

State_full_t FSM_full[4] = { // 1010 , 0110 , 0101 , 1001
    {{S1,S3},{1,1,0,0}}, // ABA'B'
    {{S2,S0},{0,1,1,0}},
    {{S3,S1},{0,0,1,1}},
    {{S0,S2},{1,0,0,1}}
};

// -----
// HALF stepping sequence - Mealy FSM

```

```

// Eight states (single/dual coil alternation) for finer step angle.
// Same encoding as FULL: next[] = next state by dir, out[] = coil pattern.
// -----
typedef struct {
    uint32_t next[2];
    uint8_t out[4];
} State_half_t;

State_half_t FSM_half[8] = {    // 1000 , 1010 , 0010 , 0110 , 0100 , 0101,
0001, 1001
    {{S1,S7},{1,0,0,0}},
    {{S2,S0},{1,1,0,0}},
    {{S3,S1},{0,1,0,0}},
    {{S4,S2},{0,1,1,0}},
    {{S5,S3},{0,0,1,0}},
    {{S6,S4},{0,0,1,1}},
    {{S7,S5},{0,0,0,1}},
    {{S0,S6},{1,0,0,1}}
};

// -----
// Stepper_init
// Maps user-provided pins to coils in order [A, B, a, b] and configures them
// as push-pull, no-pull, high-speed digital outputs.
// -----
void Stepper_init(PinName_t pinName1, PinName_t pinName2, PinName_t pinName3,
PinName_t pinName4){

    // Store pin mapping to match FSM out[] indexing: [A, B, a, b]
    myStepper.pin1 = pinName1;
    myStepper.pin2 = pinName2;
    myStepper.pin3 = pinName3;
    myStepper.pin4 = pinName4;

    // Configure pin1 (A)
    GPIO_init(myStepper.pin1,OUTPUT);
    GPIO_pupd(myStepper.pin1,nopupd);
    GPIO_otype(myStepper.pin1,pushpull);
    GPIO_ospeed(myStepper.pin1,highspeed);

    // Configure pin2 (B)
    GPIO_init(myStepper.pin2,OUTPUT);
    GPIO_pupd(myStepper.pin2,nopupd);
    GPIO_otype(myStepper.pin2,pushpull);
    GPIO_ospeed(myStepper.pin2,highspeed);

```



```

// Configure pin3 (a)
GPIO_init(myStepper.pin3, OUTPUT);
GPIO_pupd(myStepper.pin3, nopupd);
GPIO_otype(myStepper.pin3, pushpull);
GPIO_ospeed(myStepper.pin3, highspeed);

// Configure pin4 (b)
GPIO_init(myStepper.pin4, OUTPUT);
GPIO_pupd(myStepper.pin4, nopupd);
GPIO_otype(myStepper.pin4, pushpull);
GPIO_ospeed(myStepper.pin4, highspeed);
}

// -----
// Stepper_pinOut
// Drives the four coil pins according to the FSM output for the given state.
// - mode == FULL: uses FSM_full[state].out[]
// - mode == HALF: uses FSM_half[state].out[]
// Each GPIO_write sets a coil ON(1) or OFF(0) through the external driver.
// -----
void Stepper_pinOut (uint32_t state, uint32_t mode){
    if (mode == FULL){        // FULL mode
        GPIO_write(myStepper.pin1, (FSM_full[state].out[0])); // Coil A
        GPIO_write(myStepper.pin2, (FSM_full[state].out[1])); // Coil B
        GPIO_write(myStepper.pin3, (FSM_full[state].out[2])); // Coil a
        GPIO_write(myStepper.pin4, (FSM_full[state].out[3])); // Coil b
    }
    else if (mode == HALF){    // HALF mode
        GPIO_write(myStepper.pin1, (FSM_half[state].out[0])); // Coil A
        GPIO_write(myStepper.pin2, (FSM_half[state].out[1])); // Coil B
        GPIO_write(myStepper.pin3, (FSM_half[state].out[2])); // Coil a
        GPIO_write(myStepper.pin4, (FSM_half[state].out[3])); // Coil b
    }
}

// -----
// Stepper_setSpeed
// Converts rpm (rev/min) to per-step delay in milliseconds:
//   ms_per_rev = 60000 / rpm
//   step_delay = ms_per_rev / step_per_rev
// Here, step_per_rev is the chosen basis (2048) used for timing.
// -----
void Stepper_setSpeed (long whatSpeed){        // rpm [rev/min]

```

```

uint32_t steps = step_per_rev;           // half-step basis (as defined)
uint32_t ms_per_rev = 60000UL / whatSpeed; // time for one revolution [ms]
step_delay = ms_per_rev / steps;          // delay per FSM step [ms]
}

// -----
// Stepper_step
// Executes 'steps' FSM transitions in the requested mode and direction.
// Loop body per step:
// 1) compute next state via FSM table (depends on mode and direction)
// 2) output the coil pattern for that state
// 3) delay by step_delay to set the step frequency
// 'state' starts at S0 (arbitrary but consistent).
// -----
void Stepper_step(uint32_t steps, uint32_t direction, uint32_t mode){
    uint32_t state = 0;
    myStepper._step_num = steps;

    for(; myStepper._step_num > 0; myStepper._step_num--){ // run for 'steps'
iterations
        // YOUR CODE                                     // delay (step_delay);
        if (mode == FULL)
            state = FSM_full[state].next[direction]; // state = next state
        else if (mode == HALF)
            state = FSM_half[state].next[direction]; // state = next state
        Stepper_pinOut(state, mode);                  // drive coils for current
state
        delay_ms(step_delay);                          // enforce inter-step timing
    }
}

// -----
// Stepper_stop
// Immediately stops stepping and de-energizes all coils.
// Useful to drop holding torque and reduce heat/power.
// -----
void Stepper_stop (void){
    myStepper._step_num = 0;
    GPIO_write(myStepper.pin1,0);
    GPIO_write(myStepper.pin2,0);
    GPIO_write(myStepper.pin3,0);
    GPIO_write(myStepper.pin4,0);
}

```

Note that these are **blocking** stepper controllers.

While the stepper is running, the MCU cannot process other polling commands. If you can, modify it to be the non-blocking controller.

You can also create your own functions different from the given instructions.

Procedure

1. Create a new project under the directory `\repos\EC\LAB\LAB_Stepper_Motor`
 - The project name is “**LAB_Stepper_Motor**”.
 - Create a new source file named as “**LAB_Stepper_Motor.c**”

You MUST write your name on the source file inside the comment section.

2. Include your updated library in `\repos\EC\lib\` to your project.
 - **ecGPIO.h, ecGPIO.c**
 - **ecRCC.h, ecRCC.c**
 - **ecEXTI.h, ecEXTI.c**
 - **ecSysTick.h, ecSysTick.c**
 - **ecStepper.h ecStepper.h**
3. Connect the MCU to the motor driver and the stepper motor.
4. Find out the number of steps required to rotate 1 revolution using Full-steppping.
5. Then, rotate the stepper motor 10 revolutions with 2 rpm. Measure if the motor rotates one revolution per second.
6. Repeat the above process in the opposite direction.
7. Increase and decrease the speed of the motor as fast as it can rotate to find the maximum and minimum speed of the motor.
8. Apply the half-stepping and repeat the above.

Configuration

Digital Out	SysTick
PB10, PB4, PB5, PB3 NO Pull-up Pull-down Push-Pull Fast	delay()

Algorithm

Overview

Mealy FSM Table — Stepper Motor (A=PB_10, B=PB_4, a=PB_5, b=PB_3)

FULL-Step (4 States)

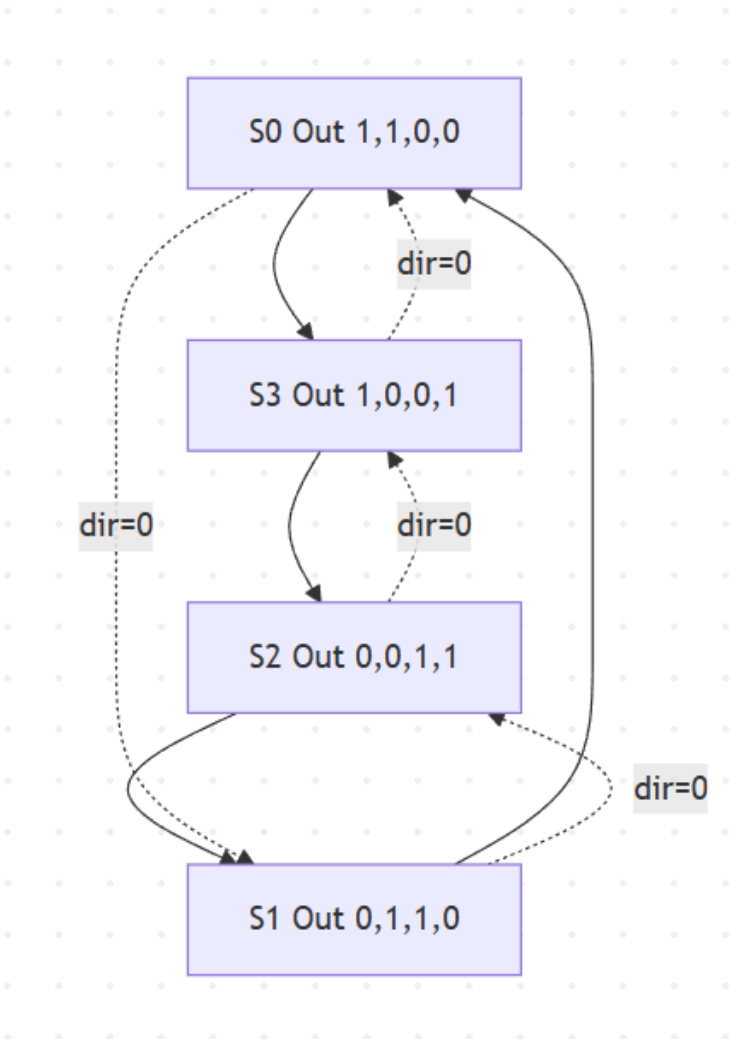
State	next[dir=0]	next[dir=1]	Out [A, B, a, b]
S0	S1	S3	1, 1, 0, 0
S1	S2	S0	0, 1, 1, 0
S2	S3	S1	0, 0, 1, 1
S3	S0	S2	1, 0, 0, 1

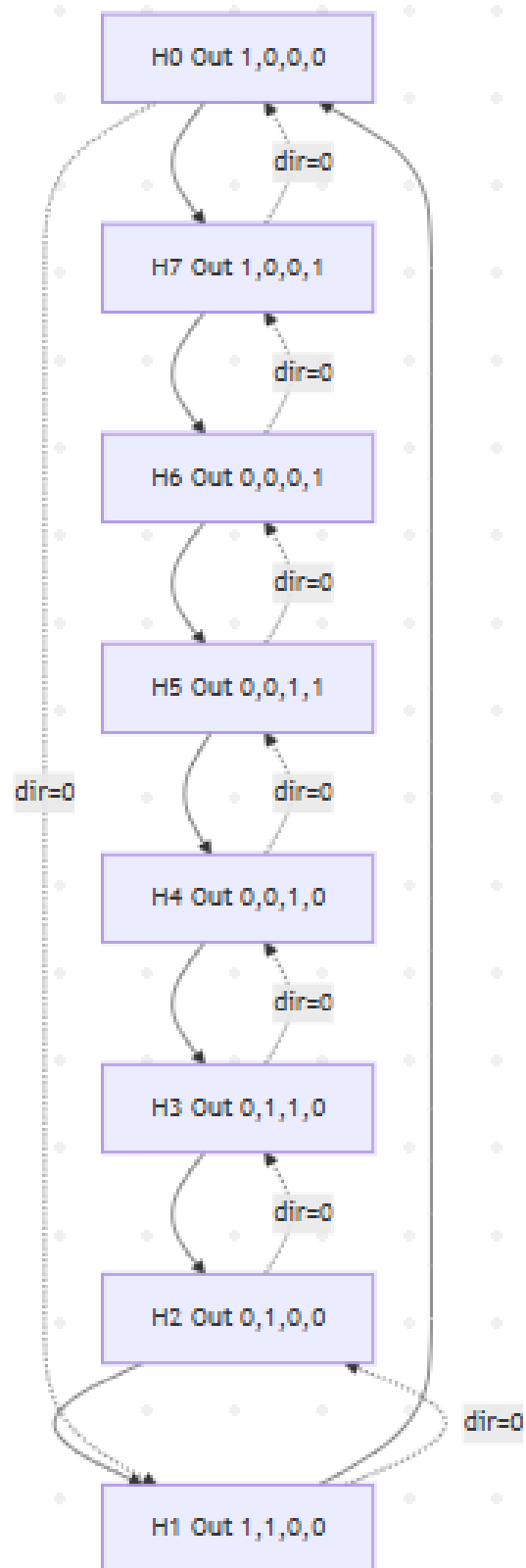
HALF-Step (8 States)

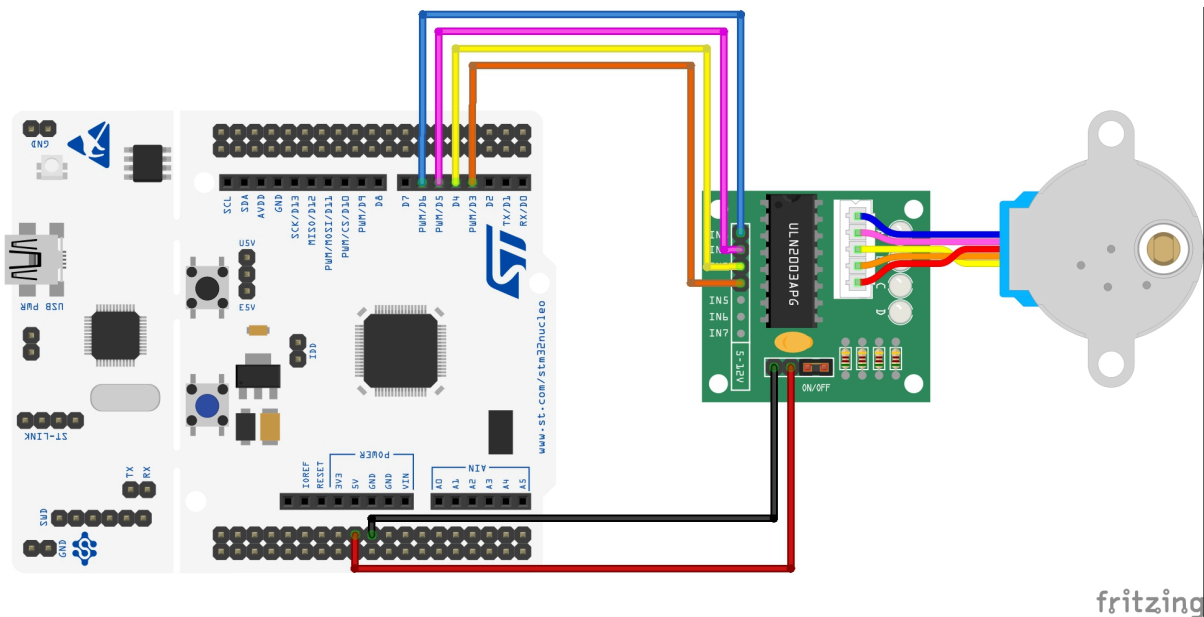
State	next[dir=0]	next[dir=1]	Out [A, B, a, b]
S0	S1	S7	1, 0, 0, 0
S1	S2	S0	1, 1, 0, 0
S2	S3	S1	0, 1, 0, 0
S3	S4	S2	0, 1, 1, 0
S4	S5	S3	0, 0, 1, 0
S5	S6	S4	0, 0, 1, 1
S6	S7	S5	0, 0, 0, 1

State	next[dir=0]	next[dir=1]	Out [A, B, a, b]
S7	S0	S6	1, 0, 0, 1

Flowchart – Full Step







Discussion

1. Find out the trapezoid-shape velocity profile for a stepper motor. When is this profile necessary?

What it is

- Speed–time curve with three phases: **accelerate** → **constant speed** → **decelerate** (trapezoid shape).

Why/when it's necessary

- To **prevent missed steps** in open-loop steppers when:
 - **High target speed** (cannot jump to it instantly).
 - **High inertia loads** (lead screw, heavy axes, gear trains).
 - **Precision motion** (CNC, 3D printers, pick-and-place) where smoothness matters.
 - **Resonance zones** need to be crossed gradually.

Key idea

- Increase step frequency linearly up to f_{target} , hold, then decrease linearly to 0.
- Typical jerk-limited alternative: **S-curve** (adds smooth accel/decel to reduce vibration even more).

Mini spec

- Let total steps = N , accel steps = N_{acc} , decel steps = N_{dec} , cruise steps = N_{cruise} .
- Step interval during accel (ms/step): $T_k = T_0 - k \cdot \Delta T$ (linear approximation) until f_{target} .
- Choose N_{acc} , N_{dec} so motor torque > load torque throughout.

2. How would you change the code more efficiently for micro-stepping control? You don't have to code this but need to explain your strategy.

Goal: Create smooth intermediate electrical angles by **controlling coil currents as sin/cos** rather than ON/OFF.

Core strategy

1. **Use PWM current control** for each coil channel (H-bridge or dedicated driver with current regulation).
2. **Lookup tables:**
 - Precompute `sin( $\theta$ )` and `cos( $\theta$ )` arrays scaled to PWM range (e.g., 0–1000).
 - Table size = microsteps per electrical cycle (e.g., 64, 128...).
3. **Timer interrupt scheduler:**
 - On each microstep “tick”:
`IA_duty = LUT_sin[idx]` , `IB_duty = LUT_cos[idx]` , `idx = (idx +  $\Delta$ ) % L`
 - Δ controls speed; adjust `tick` period for trapezoid profile (accel/cruise/decel).
4. **Abstractions:**
 - `setMicrostepResolution(m)` : rebuild (or index) appropriate LUT.
 - `setTargetSpeed(v)` : converts to tick period; tie into trapezoid planner.
 - `setDirection(dir)` : reverse LUT index progression.

Why this is efficient

- **Constant-time ISR** (table lookup + duty update), minimal branching.
- **Scales** to 1/4, 1/8, 1/16... microsteps by swapping LUT or index stride.
- **Smooth torque** and **low vibration** compared to GPIO phase toggling.

Hardware note

- Requires **bipolar driver with current regulation** (A4988/DRV8825/DRV8834/TB6600, or custom H-bridge).
 - **ULN2003 (Darlington)** alone can't regulate current; it's suited to full/half-step unipolar drive.
2. There are other types of Stepper Motor Drivers that are simple to use, such as you only give one pulse signal and direction, instead of giving 4 pulse signals. Such examples are A4988, DRV 8834, and TB6600 drivers. Compare these motor drivers with ULN2003 in terms of operating method.

Feature	A4988 / DRV8834 / TB6600	ULN2003
Control interface	STEP/DIR : 1 pulse = 1 (micro)step; DIR sets direction	4 coil lines from MCU (IN1–IN4 sequence)
Micro-stepping	Built-in (typically up to 1/16–1/32)	Not built-in (full/half via GPIO sequencing only)
Current control	Yes (chopper / PWM, set via potentiometer or registers)	No (current set by coil R and supply V)
Torque smoothness	High (sinusoidal current shaping)	Lower; step torque ripple and more resonance
MCU pin count	Low (2 pins + MS pins for resolution)	Higher (4 pins per motor + timing FSM)
Protections	Over-current/thermal/UVLO (varies by chip)	None intrinsic (it's just a transistor array)
Motor type	Bipolar steppers (or unipolar as bipolar)	Unipolar steppers (e.g., 28BYJ-48)
Typical use	CNC, 3D printers, robotics, smooth precision motion	Simple/low-cost stepper tasks

Bottom line

- **A4988/DRV8834/TB6600**: simpler MCU interface, smoother motion, current limiting built-in → best for micro-stepping and higher performance.
- **ULN2003**: cheapest/simple transistor sink for unipolar coils; needs full sequence logic from MCU; limited smoothness.

Key Takeaways

- Use a **trapezoid (or S-curve) profile** whenever load inertia or high speed can cause missed steps.
- For **micro-stepping**, drive **coil currents via PWM** using **sin/cos LUT** in a **timer ISR**, and layer a **trapezoid planner** on top.
- Prefer **STEP/DIR drivers** (A4988/DRV8834/TB6600) over **ULN2003** when you need smoothness, micro-stepping, or simpler firmware. Answer discussion questions

Code

Your code goes here: https://github.com/henny041520-commits/EC-DJLee-042/blob/main/lab/LAB_Stepper_Motor/LAB_Stepper_Motor.c

Explain your source code with necessary comments.

```
#include "stm32f411xe.h"
#include "ecGPIO2.h"
#include "ecRCC2.h"
#include "ecEXTI2.h"
#include "ecSysTick2.h"
#include "ecStepper2.h"

// -----
// Pin mapping for stepper coils
// A, B, a, b correspond to motor coil control lines connected to ULN2003
// driver.
// -----
#define A PB_10
#define B PB_4
#define a PB_5
#define b PB_3

// -----
// Defined speed limits (in rpm) based on stable operation range
// FULL-step : 1-14 rpm
// HALF-step : (1-29)/2 rpm
// -----
#define minspeed 1
#define maxspeed 14
#define hminspeed 1
#define hmaxspeed 29

// Array to simplify pin initialization loop
PinName_t step_pin[4] = { PB_10, PB_4, PB_5, PB_3 };

// -----
// Function prototype
// -----
void setup(void);

int main(void) {
    // -----
    // Initialization sequence
    // -----
    setup();
}
```

```

// ----- FULL-step tests -----
Stepper_setSpeed(2); // set speed = 2 rpm
Stepper_step(2048*10, 1, FULL); // rotate 10 rev CW (dir=1)
delay_ms(500);

Stepper_setSpeed(2); // set speed = 2 rpm
Stepper_step(2048*10, 0, FULL); // rotate 10 rev CCW (dir=0)
delay_ms(500);

Stepper_setSpeed(minspeed); // minimum stable speed (1
rpm)
Stepper_step(2048, 1, FULL); // rotate 1 rev CW
delay_ms(500);

Stepper_setSpeed(maxspeed); // maximum stable speed (14
rpm)
Stepper_step(2048, 1, FULL); // rotate 1 rev CW
delay_ms(500);

// ----- HALF-step tests -----
Stepper_setSpeed(4); // set speed = 2 rpm
Stepper_step(2048*2, 1, HALF); // rotate 2/2 rev CW
delay_ms(500);

Stepper_setSpeed(4); // set speed = 2 rpm
Stepper_step(2048*20, 0, HALF); // rotate 20/2 rev CCW
delay_ms(500);

Stepper_setSpeed(hminspeed); // half-step minimum speed (0.5
rpm)
Stepper_step(2048*2, 1, HALF); // rotate 2/2 rev CW
delay_ms(500);

Stepper_setSpeed(hmaxspeed); // half-step maximum speed (14.5
rpm)
Stepper_step(2048*2, 1, HALF); // rotate 2/2 rev CW
delay_ms(500);

// Infinite loop - keeps program running after tests
while(1){
    ; // do nothing
}
}

// -----
// setup()

```

```

// Initializes system clock, SysTick timer, and stepper GPIO pins.
// -----
void setup(void){

    RCC_PLL_init();                // configure PLL: system
    clock = 84 MHz
    SysTick_init();                // initialize SysTick timer
    (for delay_ms)

    // GPIO configuration for all stepper pins
    for(int i = 0; i <= 3; i++) {
        GPIO_init(step_pin[i], OUTPUT);    // set pin as output
        GPIO_ospeed(step_pin[i], highspeed); // set high output speed
        GPIO_otype(step_pin[i], pushpull);  // push-pull output type
        GPIO_pupdp(step_pin[i], nopupdp);   // no pull-up / pull-down
    }

    // initialize stepper motor control pins with sequence A,B,a,b
    Stepper_init(PB_10, PB_4, PB_5, PB_3);
}

```

Sample Code : Stepper Motor

```

#include "stm32f411xe.h"
#include "ecGPIO.h"
#include "ecRCC.h"
#include "ecEXTI.h"
#include "ecSysTick.h"
#include "ecStepper.h"

void setup(void);

int main(void) {
    // Initialiization -----
    -
    setup();

    Stepper_step(2048, 1, FULL); // (Step : 2048, Direction : 0 or 1, Mode :
    FULL or HALF)

    // Inifinite Loop -----
    --
    while(1){;}
}

```

```

// Initialiization
void setup(void){

    RCC_PLL_init();                // System Clock = 84MHz
    SysTick_init();                // SysTick init

    EXTI_init(BUTTON_PIN, FALL, 0); // External Interrupt Setting
    GPIO_init(BUTTON_PIN, EC_DIN);  // GPIOC pin13 initialization

    Stepper_init(PB_10, PB_4, PB_5, PB_3); // Stepper GPIO pin initialization
    Stepper_setSpeed(2);                // set stepper motor
    speed
}

void EXTI15_10_IRQHandler(void) {
    if (is_pending_EXTI(BUTTON_PIN)) {
        Stepper_stop();
        clear_pending_EXTI(BUTTON_PIN); // cleared by writing '1'
    }
}

```

Results

Experiment images and results

[EC 25-2] LAB Stepper_Motor_Full_Step_10 : <https://youtu.be/f22WbnkMSMU>
 [EC 25-2] LAB Stepper_Motor_Full_Step_10_reverse : <https://youtu.be/Zao6RR0aJBU>
 [EC 25-2] LAB Stepper_Motor_Full_Step_min : https://youtu.be/HYBWWg_fNro
 [EC 25-2] LAB Stepper_Motor_Full_Step_max : <https://youtu.be/KBXkITyvAHE>
 [EC 25-2] LAB Stepper_Motor_Half_Step_10 : https://youtu.be/M-bgF3o_06o
 [EC 25-2] LAB Stepper_Motor_Half_Step_10_reverse : <https://youtu.be/rKX4f4DsPn8>
 [EC 25-2] LAB Stepper_Motor_Half_Step_min : <https://youtu.be/iKn8MdpMG90>
 [EC 25-2] LAB Stepper_Motor_Half_Step_max : <https://youtu.be/Z0JvAXHSYPM>

Reference

Complete list of all references used (github, blog, paper, etc)

Troubleshooting

(Option) You can write Troubleshooting section